

SPACEGAME

by Kaloyan Milev

Table of Contents

1. Introduction.....	2
2. Goals and Objectives.....	2
2.1 Main Goal.....	2
2.2 SMART Goal.....	2
2.3 Learning Objectives.....	2
3. Scope.....	3
3.1 In Scope.....	3
3.2 Out of Scope.....	3
4. Requirements.....	3
4.1 Functional Requirements and MoSCoW Prioritisation.....	4
4.2 Technical Constraints.....	4
5. Target audience.....	5
6. Advice.....	5
7. Technical Details.....	5

1. Introduction

The project “SpaceGame” consists of developing a 2D singleplayer arcade game for the company [Dogfood Studios](#). The project is part of a 4-week ICT challenge focused on learning how to approach the production of a game. The main purpose is not to create a commercially finished game, but to develop a playable prototype. Because of the time limitation, the project will focus on core gameplay mechanics, while additional features will be added if enough time remains.

The gameplay consists of a player controlling a ship in order to survive increasingly difficult waves of asteroids coming towards him while collecting upgrades/rewards and achieving the highest possible score.

2. Goals and Objectives

2.1 Main Goal

The main goal is to develop a playable 2D single-player arcade game prototype in C using raylib, without using a complete game engine, within the four-week project period.

2.2 SMART Goal

SMART criterion	Application to SpaceGame
Specific	Develop a 2D arcade game in which the player controls a spaceship, avoids and destroys asteroids, earns score and rewards, and survives increasing difficulty waves.
Measurable	The project is successful when a first-time player can complete one full gameplay loop. The gameplay loop consists of: starting a run, controlling the ship, avoiding and shooting asteroids, seeing the score change, losing when hit, restarting/closing the game.
Achievable	I already have experience with C and Embedded C. The project deliberately uses a lightweight library instead of a full game engine so that the scope remains manageable.
Relevant	The project is intended to demonstrate my current programming level and help me improve in C, raylib, Vim and independent technical learning.
Time-bound	The playable prototype must be completed within the four-week project period.

2.3 Learning Objectives

1. Understand my current C level and improve my practical C programming skills.
2. Learn how to use [raylib](#) for window management, input, textures, rendering, audio and other game-related functionality.

3. Learn how to use Vim effectively as the primary editor for a larger programming project.
4. Improve my ability to learn new technologies mainly through official documentation, datasheets and technical forums, while keeping AI usage minimal.
5. Learn the fundamental structure of a real-time game, including the game loop, input handling, rendering, collision detection, object management and game states.

3. Scope

The scope is limited by the four-week development period. The main priority is therefore to create a stable and playable core game before adding optional features.

3.1 In Scope

- A small playable single-player game prototype.
- A 2D space environment.
- Player-controlled spaceship movement.
- Moving objects, such as asteroid and rewards.
- Shooting and destroying asteroids.
- Basic collision handling.
- A score system.
- Difficulty that increases during a run.
- Basic game states such as gameplay, game over and restart.

3.2 Out of Scope

- A fully developed commercial game.
- Multiplayer functionality.
- Online accounts or online leaderboards.
- Story or campaign.
- Commercial publication in stores.
- Long-term maintenance, regular updates or live-service features.
- A large amount of content such as many ships, enemies or levels.
- Development of a custom game engine.

4. Requirements

The requirements are written in a priority order, MoSCoW prioritisation is used.

4.1 Functional Requirements and MoSCoW Prioritisation

Requirement	Priority
The player must be able to control the spaceship with a keyboard.	Must

Requirement	Priority
Asteroids must spawn and move from top to bottom.	Must
The player must be able to fire projectiles.	Must
Projectiles must be able to destroy asteroids when they collide.	Must
Collision detection between different structures	Must
The game must end when the player is hit by an asteroid.	Must
The player must be able to restart after losing.	Must
The game must track and display the player's score.	Must
Asteroids should become faster when the score is higher.	Must
Game Over Menu showing Score and Highscore	Should
Temporary upgrades or rewards should appear during gameplay.	Should
The game should contain variation in asteroid behaviour, durability, speed or appearance.	Should
The game should contain a basic main menu.	Should
The player could unlock additional ships.	Could
The game could have different game modes.	Could
Different ships could have different shooting behaviour.	Could
The game could contain music, sound effects and settings for brightness/SFX/music.	Could
The project will include multiplayer functionality.	Won't

4.2 Technical Constraints

Constraint
The game will be written only in C.
raylib will be used as the main multimedia library.
No complete game engine such as Unity, Unreal or Godot will be used.
Vim will be used as the code editor.

5. Target audience

“SpaceGame” is mainly targeted towards people between 10 and 20 years old, who enjoy simple arcade style games. The game is also for people, who like shooting, avoiding obstacles and chasing the highest score possible.

The secondary target audience is older people, who feel nostalgia about the retro arcade games from the 1980's.

6. Advice

The game is fairly lightweight, because of its simplicity, but using a dedicated PC is advised. A dedicated PC is one that has a monitor with a resolution of at least 800x500 pixels, a keyboard with a proper layout and enough storage for the game and its assets(such as .png files).

Another recommendation is distributing the game as a compiled release.

7. Technical Details

Technical details such as comparisons between technologies and reasons for specific technical choices are written in the “[Technical Details](#)” Document.